Code Coverage-Guided Fuzzing

Contents

- 9.1. Learning outcomes of this chapter
- 9.2. Chapter Introduction
- 9.3. Code Coverage-Guided Fuzzing

9.1. Learning outcomes of this chapter

At the end of this chapter, you should be able to:

- Explain in general terms how coverage-guided greybox fuzzers like AFL operatate and their advantages and disadvantages as compared to random, mutation and generation-based fuzzing.

9.2. Chapter Introduction

The three fuzzing techniques we discussed in the previous chapter (random, mutation and generation-based black-box fuzzing) all suffer from the same drawback: the strategy that they use to generate inputs for the program under test has no feedback loop. Each of these methods generates inputs for the program under test but it does so in a way that is generally blind to (or unaware of) what the program is doing. For this reason all of the three techniques we studied so far are called black-box techniques, because they operate in total ignorance of the program-under-test (although they might have some knowledge of its input format, they do not have any knowledge of the program's code).

In this chapter, we will look at code coverage-guided fuzzing technique and its embodiment in a popular general-purpose fuzzer: American Fuzzy Lop (AFL).

9.3. Code Coverage-Guided Fuzzing

Recall the following program from earlier in the previous chapter, and the difficulty of reaching line 7 (where the simulated fault is). Doing so requires being able to generate the exact four bytes corresponding to the string "bad!": one choice out of a possible 2^{32} (about 4.3 billion).

```
1. void good_bad(char s[4]) {
2.   int count = 0;
3.   if (s[0] == 'b') count++;
4.   if (s[1] == 'a') count++;
5.   if (s[2] == 'd') count++;
6.   if (s[3] == '!') count++;
7.   if (count > 3) abort();  //fault
8. }
```

One way we can overcome the limitations of black-box fuzzing techniques is by allowing the fuzzer to monitor

Skip to main content

generate next. One popular technique is to monitor the code coverage achieved by each input. This technique is known as coverage-guided fuzzing. It is also called a greybox fuzzing technique because, unlike black-box techniques, it operates with partial knowledge of the program-under-test, specifically by being able to observe what parts of the program are covered (i.e. executed) by each input. The tool American Fuzzy Lop (AFL) popularised coverage-guided fuzzing, and has dominated much fuzzing research since its release in 2013. Its main ideas have been incorporated into the libFuzzer tool, which is a part of the LLVM Clang C compiler, and allows automatic coverage-guided fuzzing of programs and libraries.

The basic way that a coverage-guided fuzzer like AFL or libFuzzer operates is that it begins with some seed inputs (or by generating an initial random input). It maintains a list of interesting inputs and generates new inputs for the program-under-test by selecting an input from this list and mutating it. When running the program on this new input, it monitors the program to determine what execution path the input followed. If a new part of the program was executed that hasn't yet been executed by previous inputs, the new input is considered interesting and is added to the list of interesting inputs. Otherwise it is discarded, because it did not allow the fuzzer to learn any new information about the program-under test.

Algorithm

1. $interesting \leftarrow initial_test_seeds$
2. $seen \leftarrow \emptyset$
3. while true do
4. Choose an input i from interesting
5. $input \leftarrow MUTATE(i)$
6. $path \leftarrow EXECUTE(input)$
7. if $\{path\} \not\subset seen$ then
8. $interesting \leftarrow interesting \cup \{input\}$
9. $seen \leftarrow seen \cup \{path\}$

Note that this is a highly simplified view of the process. In reality, various heuristics are used to choose which input i should be mutated at each iteration and the specific mutation strategy to apply to i. Also, rather than tracking the precise path that the execution follows, for performance reasons, most coverage-guided fuzzers track more coarse-grained information (such as which branches in the program were taken, with an approximate count of how many times each was taken, and so on).

9.3.1. Example

Let us consider how such a fuzzer would perform on the good bad program above. Suppose it starts with the input string "good" as the initial seed input. It will immediately run the progrma on that input and discover a new path (since this is the first time the program is being run). Further execution of the program on any input that follows this same path will be considered uninteresting.

Suppose the fuzzer is very naive and mutates the input string by choosing one byte at random and mutating that byte to some randomly chosen byte. There is a 1 in four chance that the first byte is chosen and a 1 in 256 chance that the mutation produces the byte "b". Therefore there is a 1 in 1024 chance that the mutation will produce the input "bood" that causes the first if-test to succeed (on line 3). After generating about 700 inputs, then, the probability of mutating the first byte to become "b" exceeds 50%. (We will learn how to analyse such probabilities later when we discuss Markov Models in the context of Software Reliability.)

Skip to main content

As soon as this happens, the fuzzer will notice that a new execution path was followed and the new input (e.g. “bood”) will be added to the list of interesting inputs. Following a similar argument, the input “bood” needs to be mutated only about 700 times before the probability of generating the input “baod” exceeds 50%, uncovering the second branch and adding a third interesting input.

This argument can be repeated to consider all of the ways in which the ultimate input “bad!” might be progressively discovered by a coverage-guided fuzzer. Doing so reveals that within some thousands of mutations we can expect even a naive coverage guided fuzzer to uncover the fault in this program.

This is a massive advantage compared to other fuzzing methods. For this reason coverage guided fuzzing has become a very popular general purpose fuzzing technique. Coverage-guided greybox fuzzers like AFL and lib-Fuzzer have been used to find many security vulnerabilities across a range of programs. They are especially good at reaching deep program paths that involve incremental equality tests like the good bad program above.

9.3.2. The Problem of Multi-Byte Equality Tests

However, suppose we rewrote the good bad program to perform the entire four-byte comparison of the array s against the string “bad!” in a single action, e.g. by casting both to (four-byte) integers, as shown in the following program:

```
1. void good_bad(char s[4]) {
2.   int *i = (int *)s;
3.   const char t[4] = {'b','a','d','!'};
4.   int *j = (int *)t;
5.   if (*i == *j) {
6.     abort();  //fault
7.   }
8. }
```

This program also has a fault when the supplied string is “bad!”. But now, instead of having 16 different paths (as the good bad program does) that allow the coverage-guided fuzzer to incrementally discover the needed input one-byte-at-a-time, it now only has two paths. Thus a coverage-guided fuzzer is not given any feedback until it happens to discover the input “bad!” which, recall, is incredibly unlikely. In this case we can expect a coverage-guided fuzzer ot perform no better than a random fuzzer since its feedback mechanism offers it no additional information: all inputs given to this program follow the same path except the single input “bad!”.

For this reason, coverage-guided fuzzers can struggle to generate inputs that cause checksum tests and other multi-byte equality tests to succeed. One way to work around this is to rewrite such checks to be performed incrementally (one-byte-at-a-time), or to remove the check altogether. Both techniques have been used to aid coverage-guided fuzzing to find real security vulnerabilities.

Another approach is to combine coverage-guided fuzzing with symbolic execution, which is another test generation technique that we discuss later in the subject.

9.3.3. Performance Cost

In order to monitor the program-under-test to determine which path an execution follows, one of two strategies is typically used.

- If source code for the program-under-test is available, then one approach is to compile it with a special compiler that adds extra instrumentation code to the compiled object (binary) code. This instrumentation

Skip to main content

- Or, if source code is unavailable, the program must be run in a virtual machine or an emulator (i.e. it has to be interpreted, rather than executed natively), to allow its execution to be tracked.

Both of these choices impose a performance overhead. The first is generally faster and so is preferable when source code is available. However the latter has the advantage of being applicable to privileged code like operating system kernels which are otherwise difficult to fuzz.

9.3.4. Advantages

- Good coverage can be achieved even with little to no knowledge of the input format
- Low set-up time
- The technique can be widely applied

9.3.5. Disadvantages

- Coverage can still be limited in programs that perform multi-byte equality tests (e.g. that compare two 4-byte integers), since such comparisons provide no incremental feedback to the fuzzer to allow it to discover the needed input to pass the test.
- Performance overhead is higher than with fuzzing techniques that use no feedback loop.
- For programs whose input format is known, generation-fuzzing and similar techniques are likely to perform better since they can avoid the drawbacks of coverage-guided fuzzing.

9.3.6. Tool support

There are many tools to support coverage-guided fuzzing.

- American Fuzzy Lop (AFL) (google/AFL) popularised coverage-guided fuzzing.
- libFuzzer (https://llvm.org/docs/LibFuzzer.html) is a coverage-guided fuzzer built into the LLVM C compiler, Clang.